Improving Tool-Call Decision Making in Small Language Models

Anirudh Poruri
aporuri@umd.edu

Mukund Shankar
mukunds@umd.edu

Abstract

Tool use is now a core capability of modern language models, but reliable tool use requires more than producing syntactically valid function calls. A model must also decide when not to call a tool: for example, when it should ask for missing information or admit that the available tools are insufficient. Prior work has focused more on function-call correctness than on this abstention-aware decision problem. We study this problem in small language models using Gemma-3-4B-it and Llama-3.2-3B-Instruct. We compare prompting, a linear probe over last-token hidden states for call vs. no-call prediction, standard post-training with supervised fine-tuning (SFT) and direct preference optimization (DPO), and our proposed Constitutional AI-style pipeline that uses rule-guided self-critique and preference optimization. We evaluate primarily on When2Call, with BFCL as a secondary benchmark for structured tool-call correctness. Our goal is to determine whether our proposed Constitutional AI-style pipeline can improve when-to-call decisions beyond what is achievable with prompting, linear probing, and standard supervised learning alone.

1 Introduction

Large language models are increasingly being deployed as tool-using agents that call APIs, query databases, and interact with external systems to retrieve information or execute actions. Toolformer showed that models can learn which APIs to call, when to call them, and what arguments to provide (Schick et al., 2023), while ReAct showed that reasoning interleaved with actions improves tool-use performance on knowledge-intensive tasks (Yao et al., 2023). At the same time, BFCL showed that function calling remains difficult to evaluate through its introduction of an AST-based framework for validating function calls across diverse real-world settings (Patil et al., 2025).

We study tool call decision making, where a model must decide whether to call a tool, ask a clarification question, abstain when available tools are insufficient, or answer directly. This problem is particularly important for small open-weight models, which benefit from external tools but are also more brittle when deciding whether tool use is necessary (Ross et al., 2025). The When2Call dataset explicitly addresses this issue and includes both supervised fine-tuning and preference-tuning splits, making it a natural dataset for post-training.

A second challenge is that improving abstention is not as simple as adding more negative examples. Ross et al. report that standard fine-tuning on abstention-focused data can improve performance on When2Call while hurting Berkeley Function Calling Leaderboard Abstract Syntax Tree (BFCL AST) performance, because the model becomes too conservative and under-calls tools. This tradeoff makes abstention-aware tool calling a useful testbed for post-training methods that aim to improve appropriate abstention rather than indiscriminate refusal.

Existing work evaluates tool use primarily in terms of whether a model produces a syntactically correct function call with appropriate arguments, with much less focus on whether a tool call is necessary at all. A model can output a perfectly formatted function call in a case where no tool should have been called at all (Ross et al., 2025). Ross et al. also mention that in practice, many failures arise from poor tool-use decisions: calling a tool when a direct answer would suffice, hallucinating a tool when the available tools are irrelevant, or failing to ask a follow-up question when a required parameter is missing. This gap

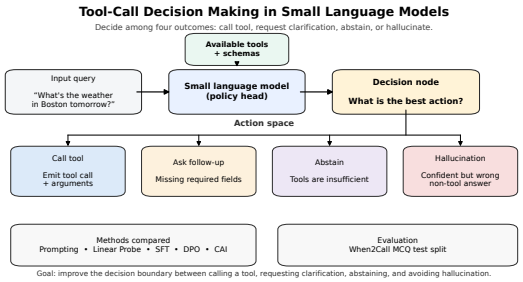

Figure 1: Abstention-aware tool use requires choosing whether to call a tool, ask for missing information, abstain because tools are insufficient, or avoid a hallucinated non-tool answer. We compare prompting, a linear probe, SFT, DPO, and a CAI-style pipeline on this decision in small language models, evaluated primarily on When2Call.

motivates our study of whether post-training methods can improve abstention-aware tool-call decisions beyond what is achievable with prompting or standard supervised learning alone. Understanding which post-training strategies improve abstention-aware tool use is important for deploying reliable small language models in tool-integrated environments.

To address this gap, we compare four approaches of increasing complexity. First, we use zero-shot and 4-shot prompting as lightweight baselines for abstention-aware decision making. Second, we train a linear probe on the last-token hidden state to predict call vs. no-call, motivated by work on selective prediction and abstention in NLP (Xin et al., 2021). Third, we use standard post-training baselines: supervised fine-tuning (SFT) with LoRA and Direct Preference Optimization (DPO), both trained on data aligned with the abstention-aware tool-use setting (Hu et al., 2021; Rafailov et al., 2023). Fourth, we study a CAI-style pipeline in which a tool-calling constitution with explicit, checkable rules guides self-critique, self-revision, and preference optimization (Bai et al., 2022; Rafailov et al., 2023). We include DPO because abstention errors are naturally expressed as preference pairs and because it avoids reward-model training and on-policy reinforcement learning under limited compute.

Our evaluation focuses on abstention. We use When2Call as the primary benchmark because it captures non-tool behaviors rather than simple relevance filtering (i.e., asking follow-up questions), and we use BFCL as a secondary benchmark to ensure that improved abstention does not come at the cost of substantially worse structured function-call correctness (Ross et al., 2025; Patil et al., 2025). Our contributions are:

1. We frame abstention-aware tool calling as a reliability problem for small open-weight language models.

2. We compare four approaches for improving call vs. no-call decisions: prompting, a linear probe over hidden activations, post-training with Supervised Fine-Tuning (SFT) and Direct Preference Optimization (DPO), and a constitutional SFT to DPO pipeline.

3. We evaluate these approaches primarily on When2Call to measure abstention-aware decision quality, BFCL to measure structured function-call correctness.

2 Related work

2.1 Tool use and function-calling benchmarks

Toolformer showed that language models can learn which APIs to call, when to call them, and what arguments to provide through self-supervised training with a small number of demonstrations per API (Schick et al., 2023). ReAct further showed that interleaving reasoning with external actions can improve decision-making and reduce hallucination in knowledge-intensive tasks (Yao et al., 2023). These works establish the value of tools, but they do not by themselves solve the problem of evaluating tool use across diverse settings. This evaluation gap motivated the Berkeley Function Calling Leaderboard (BFCL), which introduced a large-scale benchmark and an AST-based evaluation framework for checking function-call correctness across multiple tool-calling scenarios (Patil et al., 2025). BFCL spans single-turn, multi-turn, and more agentic settings, making it a useful benchmark for structured call correctness.

2.1.1 Deciding whether to use tools

Some work argues that the biggest challenge in tool use is often not just how to call a tool, but whether a tool should be called at all (Huang et al.,

2023; Ross et al., 2025). Recent benchmarks and datasets have moved beyond schema correctness and tool selection to study decision-making under tool uncertainty, including whether a model should call a tool, answer directly, ask a follow-up question, or abstain altogether.

Among these, When2Call is the closest to our experiment. It is explicitly designed around abstention-aware tool use and has four distinct behaviors: generating a tool call, producing a direct answer, asking a follow-up question, and admitting that the available tools are insufficient (Ross et al., 2025). This setup is relevant for our paper because our goal is not just to improve call formatting, but to improve the model's decision about whether tool use is appropriate in the first place. An important finding from this paper is that naive fine-tuning on abstention-oriented data can make models overly conservative on tool calls, decreasing tool-calling accuracy. This tradeoff motivates more careful post-training strategies that aim to increase appropriate abstention without suppressing necessary tool use.

Our classifier baseline is also motivated by recent work showing that internal model representations can predict downstream success before full generation is complete (Afzal et al., 2025). Azfal et al. find that hidden states on reasoning traces can reveal whether a chain-of-thought attempt is likely to succeed even before the model finishes generating. This suggests that useful decision signals may already be encoded in intermediate or final hidden states before tool use, which supports our use of a lightweight linear probe over hidden activations to predict call vs no-call.

A closely related direction is model-internal confidence estimation for tool-using agents (Subramani et al., 2025). Subramani et al. propose using intermediate-layers and their agreement with final outputs as signals for estimating confidence in tool-calling decisions. These confidence estimators have been shown to improve whether a model chooses to call a tool under different risk settings. Compared with that approach, our probe is intentionally simpler: rather than learning a more complex confidence estimator over multiple internal decoding signals, we use a transparent binary classifier over last-token representations as a baseline for abstention-aware decision making.

2.1.2 Self-critique, constitutions, and preference optimization

Another relevant line of work studies whether language models can improve their outputs through self-critique and revision (Madaan et al., 2023). Self-Refine showed that a model can iteratively generate feedback on its own output and then revise that output without additional human supervision. This idea is closely related to our experiment because tool-use decisions can often be critiqued against explicit structural rules, such as missing arguments or unjustified tool calls.

Because abstention-aware tool-use decisions can be evaluated against explicit structural rules, they are particularly well suited to rule-guided self-critique pipelines such as CAI (Bai et al., 2022). CAI uses a written constitution to guide self-critiques and self-revisions, then further improves the model using AI-generated preference data. This framework is a strong fit for abstention-aware tool calling because the behavior we care about is governed by short, explicit, and checkable rules, such as abstaining on irrelevance, asking follow-up questions when inputs are missing, and avoiding hallucinated arguments. Unlike the original CAI setting, which focused on harmlessness and helpfulness in dialogue, our constitution is task-specific and targeted at a constrained tool-use decision problem.

For preference optimization, we use DPO (Rafailov et al., 2023). DPO replaces reward-model-based RLHF with a direct objective over preference pairs, avoiding reward-model training and on-policy reinforcement learning. This is beneficial in our setting because abstention-aware tool-use errors are naturally expressible in pairwise form: a constitution-compliant response can be ranked above one that hallucinates a tool call, invents an argument, or fails to ask a needed follow-up (Rafailov et al., 2023). When2Call further reinforces this choice because it already includes a preference-tuning split, and the authors report that preference-based training is better suited than plain SFT for preserving the balance between calling and abstaining (Ross et al., 2025).

2.2 Efficient post-training for small models

Because our experiments target small open models, we use parameter-efficient adaptation (Hu et al., 2021; Dettmers et al., 2023). LoRA adapts pretrained transformers by injecting trainable low-rank matrices while freezing the original weights, substantially reducing the number of trainable parameters (Hu et al., 2021). QLoRA extends this idea with low-bit quantization, making supervised fine-tuning practical under tighter memory budgets (Dettmers et al., 2023).

3 Method

4 Results

4.1 Evaluation

We evaluate all baselines on the When2Call MCQ test split ($n = 3652$), which measures whether a model selects the appropriate action among `tool_call`, `request_for_info`, `cannot_answer`, and `direct`. Each example provides a query, candidate tool schemas, and a gold action label.

For each example, we score candidate labels as continuations under the model and select the label with maximum conditional likelihood, as in When2Call. Because candidate answers differ in token length, we report both raw summed log-probability and byte-length-normalized log-probability. Unless otherwise noted, normalized scoring is used as the primary metric because it reduces bias toward shorter answer strings.

Performance is reported using accuracy and macro-F1. An important property of this test split is that the class `direct` has zero gold support. As a result, any prediction of `direct` is necessarily incorrect in this setting, making prediction-mix analysis informative for diagnosing class bias.

4.2 Baselines

We evaluate eight baselines formed from two small instruction-tuned model families and four adaptation strategies. Our experiments use Llama-3.2-3B-Instruct and Gemma-3-4B-it under:

1. zero-shot prompting
2. 4-shot prompting
3. SFT
4. DPO

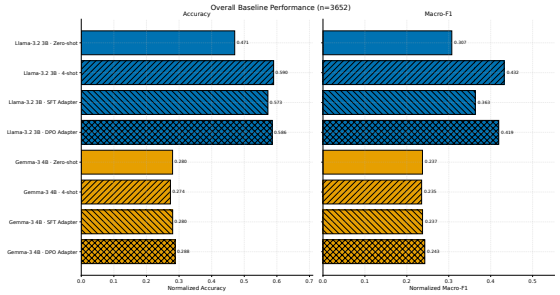

Figure 2: Normalized accuracy and macro-F1 for all eight baselines on When2Call MCQ ($n = 3652$). Llama baselines consistently outperform Gemma baselines across both metrics.

All baselines are evaluated on the same When2Call MCQ test split with a fixed candidate label set as described in earlier sections. Few-shot prompting uses four in-context examples drawn from the training split. Adapter-based baselines (SFT and DPO) use parameter-efficient LoRA-style updates so that all comparisons preserve the same base model backbone. Additionally, reporting both raw scores and byte-length normalized scores allows us to compare across approaches fairly.

4.3 Overall baseline performance

Figure 2 summarizes overall baseline quality under byte-length-normalized scoring. Across all baselines, Llama-based models consistently outperform Gemma-based models on both normalized accuracy and macro-F1.

The strongest baseline is Llama 4-shot (0.590 accuracy, 0.432 macro-F1), followed closely by Llama DPO (0.586, 0.419) and Llama SFT (0.573, 0.363). Gemma baselines cluster substantially lower (0.274–0.288 normalized accuracy), indicating comparatively limited gains from prompting and post-training in this setup.

4.4 Effect of normalization

Figures 3 and 4 isolate the effect of byte-length normalization by plotting the difference between normalized and raw scoring metrics.

Normalization produces substantial improvements for several Llama baselines but near-zero or negative changes for Gemma variants. This shows that scoring choice can materially affect

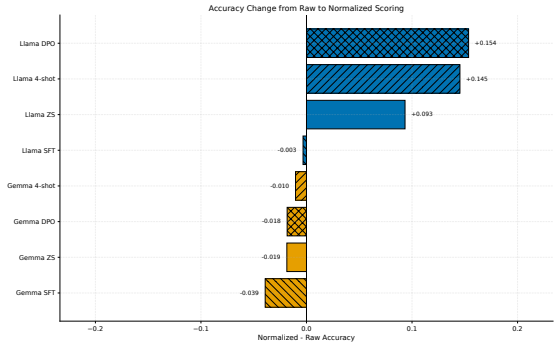

Figure 3: Change in accuracy after byte-length normalization. Normalization substantially improves several Llama baselines while having minimal effect on Gemma variants.

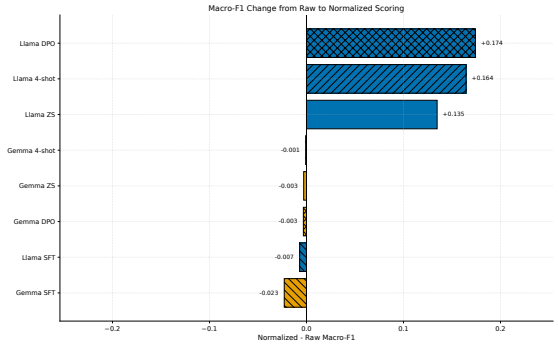

Figure 4: Change in macro-F1 after byte-length normalization. Normalization effects vary across model families and adaptation strategies.

model ranking and highlights the importance of reporting both normalized and raw views for fair comparison.

4.5 Class-level behavior

Figure 5 shows class-level recall under normalized scoring. Across baselines, the most difficult class is `request_for_info`, while recall for `cannot_answer` improves substantially after supervised fine-tuning.

Figure 6 compares predicted label distributions with the gold distribution. Several Gemma baselines allocate substantial probability mass to the unsupported class `direct`, which directly reduces performance.

Overall, these results suggest that prompting remains a strong baseline for abstention-aware decision making in small models, while adapter-based post-training changes the class distribution in different ways across model families. In particular, SFT can make models more conservative: for Llama, it sharply improves recall on `request_for_info` and `cannot_answer` but severely hurts `tool_call` recall. DPO is more stable, recovering much of the strong tool-call behavior of prompting while still improving abstention-related classes relative to zero-shot prompting. These results are consistent with prior findings that naive fine-tuning on abstention-oriented data can improve abstention while suppressing necessary tool use.

Figure 5: Class-wise recall under normalized scoring. The `request_for_info` class is the most difficult for several baselines.

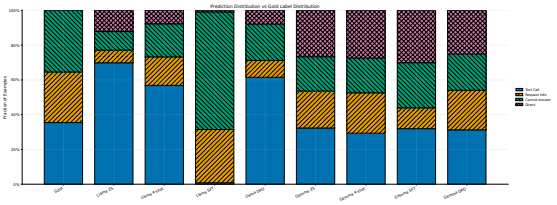

Figure 6: Prediction distribution compared with gold-label distribution. Over-prediction of the unsupported `direct` class explains part of the Gemma performance gap.

References

Afzal, A., Matthes, F., Chechik, G., and Ziser, Y. (2025). Knowing before saying: LLM representations encode information about chain-of-thought success before completion. In *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria.

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., Chen, C., Olsson, C., Olah, C., Hernandez, D., Drain, D., Ganguli, D., Li, T. B. B., Tran-Johnson, E., Perez, E., Kerr, J., Mueller, J., Ladish, J., Landau, J., Ndousse, K., Lukosiute, K., Askell, et al. (2022). Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*.

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L.

(2023). QLoRA: Efficient finetuning of quantized LLMs. *arXiv preprint arXiv:2305.14314*.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2021). LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Huang, Y., Shi, J., Li, Y., Fan, C., Wu, S., Zhang, Q., Liu, Y., Zhou, P., Wan, Y., Gong, N. Z., and Sun, L. (2023). MetaTool benchmark for large language models: Deciding whether to use tools and which to use. *arXiv preprint arXiv:2310.03128*.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., and Clark, P. (2023). Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*.

Patil, S. G., Mao, H., Yan, F., Ji, C. C.-J., Suresh, V., Stoica, I., and Gonzalez, J. E. (2025). The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*.

Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. *arXiv preprint arXiv:2305.18290*.

Ross, H., Mahabaleshwarkar, A. S., and Suhara, Y. (2025). When2Call: When (not) to call tools. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Albuquerque, New Mexico.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*.

Subramani, N., Eisner, J., Svegliato, J., Van Durme, B., Su, Y., and Thomson, S. (2025). MICE for CATs: Model-internal confidence estimation for calibrating agents with tools. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Albuquerque, New Mexico.

Xin, J., Tang, R., Yu, Y., and Lin, J. (2021). The art of abstention: Selective prediction and error regularization for natural language processing. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, Online.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. (2023). React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.